

Monash University

FIT5147 Data Exploration and Visualisation, Semester 1, 2026

Programming Exercise 3: D3 (5%)

1. Instructions and Brief

In this assignment you complete an interactive D3.js visualisation of Australian parrot occurrence data by filling in the task blocks of a partially-finished HTML template. The data comes from the Atlas of Living Australia (ALA, <https://www.ala.org.au>) and covers fifteen species of Australian parrot, with observation counts broken down by state and season. The template already loads the data for you — you do not need to fetch, process, or modify it. If you want to browse the raw data, it is available at <https://github.com/gavjmooney/fit5147-sem1-2026-pe3-data>, but this is not required to complete the assignment.

This is an individual assignment and worth 5% of your total mark for FIT5147.

Data attributes

The data is a taxonomic hierarchy — order → family → genus → species — loaded into a `d3.hierarchy` called `root`. Every node `d` exposes its underlying record at `d.data`. The fields you will use across the tasks are:

- `d.data.name` — the taxonomic name (e.g. *Psittaciformes*, *Cacatuidae*, *Cacatua galerita*). Present on every node.
- `d.data.rank` — one of "order", "family", "genus", "species". Present on every node.
- `d.data.total_observations` — integer ALA record count. On a species leaf this is its own count; on an internal node it is the sum of its descendant species.
- `d.data.commonName` — species leaves only (e.g. "Sulphur-crested Cockatoo").
- `d.data.scientificName` — species leaves only (e.g. *Cacatua galerita*).
- `d.data.observations_by_state` — species leaves only. A dictionary keyed by full state name ("Western Australia", "South Australia", "Victoria", "Tasmania", "Australian Capital Territory", "New South Wales", "Queensland", "Northern Territory") with integer record counts. Not every species has an entry for every state.

The scaffold also provides the `STATES` array (giving the left-to-right order to use across all glyphs), the `STATE_ABBR` abbreviation lookup, and the helper `familyOf(d)`, which walks from any node up to its family ancestor and returns the family name.

Relevant learning outcomes

LO6. Implement interactive data visualisations using R and other tools.

Originality

This is an individual assignment and must be your own work. Acknowledge any external source you use, including code, tutorials, designs, blog posts, Stack Overflow answers, or anything else you reuse or were influenced by. Add the acknowledgement in a code comment at the relevant location, with an APA- or IEEE-style reference.

Generative AI rules for this assessment:

- **Code (allowed).** You may use a Generative AI tool (e.g. ChatGPT, Claude, Copilot) to help you write the JavaScript, HTML, or CSS for the five D3 tasks. AI-generated code can be wrong or out of date — check and test anything you include.
- **Reflection (not allowed).** You may not use Generative AI to write, rephrase, expand, or polish any part of your reflection. The only permitted assistance is a basic spell- or grammar-checker to fix obvious errors.

If you used a Generative AI tool, declare it in the reflection PDF (see Section 7, [How to Submit](#)): name the tool, say what you used it for, and include either a shared chat link or the full list of prompts you sent. **If you did not use a Generative AI tool, state that explicitly.**

Your submission is run through similarity-detection software. Suspected breaches of academic integrity — including plagiarism, collusion, contract cheating, or undeclared use of Generative AI — will be reviewed and may result in penalties.

2. The Final Visualisation

The visualisation is a horizontal tree diagram. Specifically, it is a *tidy tree*: nodes at the same level are aligned vertically so each taxonomic rank forms a tidy column. The tree branches left-to-right starting from the order *Psittaciformes*, through family and genus, out to the fifteen species at the right edge. Because the species nodes have no children below them, they sit at the ends of the branches and are called *leaves* (the same way the tips of a real tree’s branches are leaves). Two families appear: the cockatoos (*Cacatuidae*) and the lorikeets and other Australian parrots (*Psittacidae*).

Next to each species leaf is a small bar chart with eight bars, one per state/territory, in a fixed left-to-right order: WA, SA, VIC, TAS, ACT, NSW, QLD, NT. We refer to these little per-species charts as *glyphs* — “glyph” just means a small visual mark attached to a data point. Each species’ bars are scaled to its own largest count, so the tallest bar always reaches the top of the chart. This means a glyph shows the *shape* of a species’ distribution across states (which states it is most often seen in), while the size of the species node shows the species’ *total* record count.

Hovering over any node or bar shows a tooltip with relevant detail, and the hovered node’s path back to the root and its subtree are highlighted while the rest of the tree fades out. A legend shows how families are encoded and a sample of node sizes.

3. The Template

You are provided with a single HTML file, `template.html`, that includes the page layout, CSS, the title, the data source reference, and a scaffolded JavaScript block. The scaffold already does the following on your behalf:

- Loads the data and constructs the d3 hierarchy, applying a `d3.cluster()` layout so every node has `d.x` and `d.y` positions.
- Draws the tree links between nodes.
- Creates an empty `<g>` element at each node’s position (the `node` selection) and a convenience selection of just the species leaves (`leafSel`).
- Defines `familyOf(d)`, a helper that walks from any node up to its family ancestor and returns the family name.

- Provides constants for the bar chart layout: `STATES`, `STATE_ABBR`, `GLYPH_OFFSET`, `GLYPH_W`, `GLYPH_H`, `BAR_WIDTH`.
- Renders axis labels: rank column headers (Order, Family, Genus, Species) and state abbreviation headers above the first species' bar chart.
- Sets up a legend group with section headings ("Family", "Bar glyph" with explanation text, "Number of observations") — but the visual entries beneath each heading are left for you to complete.
- Renders the data-source footer.

Your job is to complete five numbered task blocks inside the script. Each block begins with `// #TASK N` and ends with `// #END TASK N`. All of your code belongs inside these blocks. Do not modify the scaffold outside them.

4. Tasks

Task 1: Draw and style the nodes.

- 1.1 Create a visual marker at each node.
- 1.2 Encode each node's family using a visual variable of your choice, so that the two parrot families are visually distinct. Family, genus and species nodes should reflect their family; the order root, which belongs to neither family, should be visually distinguishable from both.
- 1.3 Make each node's size proportional to the number of observations at that node. Observation counts span several orders of magnitude, so choose a scale that keeps both rare and common species readable.
- 1.4 Label the nodes. Internal nodes (order, family, genus) should display their taxonomic name. Species leaves should show both the common name and the scientific name.

Task 2: Draw a small bar chart next to each species. Add a bar chart at every species leaf with one bar per state/territory (eight bars total). Use the same left-to-right state order for every species so the charts can be compared — the `STATES` array gives you that order. Within each species, scale the bar heights so the species' largest count fills the chart (i.e. its tallest bar reaches the top); this lets the chart show the *shape* of the species' distribution across states/territory, since the species' total count is already shown by node size. Set a `data-state` attribute on each bar to its state name — you will use it in Task 3.2 to identify which bar the cursor is on.

Task 3: Tooltip on hover. Show a tooltip that follows the cursor when the user hovers over elements in the tree. Position the tooltip near the cursor and ensure it is entirely viewable.

- 3.1 When hovering over a node, show the node's name and its total observation count. For species leaves, show both the common name and scientific name.
- 3.2 When hovering over a bar in a species' bar chart, show the species' common name, state/territory name, and that state/territory observation count for the species.

Task 4: Highlight on hover. When the user hovers over a node, visually highlight its context in the tree. The highlighted set should include the hovered node, its ancestors (the path back to the root) and its descendants (everything below it, including the bar glyphs). The highlight should reset when the cursor leaves the node. How you visually distinguish the highlighted elements from the rest of the tree is your design choice.

Task 5: Complete the legend. The scaffold provides a legend with section headings but no visual entries beneath them. Fill these in to reflect your design choices.

5.1 Under “Family”, add an entry for each family showing the encoding you used in Task 1.2, with a label identifying the family.

5.2 Under “Number of observations”, add sample markers at a few representative sizes so the reader can gauge the node-size scale.

Detailed per-task instructions are included inline in `template.html`.

5. Reflection

In addition to the D3 visualisation, you must write a short reflection (**max 300 words**) answering the following question:

You have now completed three visualisation programming exercises on Australian parrot data, using Tableau, R, and D3. Although the visualisations and datasets differed across the three, reflect on your experience using each tool and answer the following: For each tool, identify one thing it made easy and one thing it made hard. Refer to specific design or implementation decisions from your submissions (for example, how you encoded categorical variables, handled scales, or implemented interactivity). Which tool would you choose for a similar task in future, and why?

The reflection is submitted as a PDF, separately from your code (see Section 7). Generative AI must not be used to write or rephrase any part of the reflection — see the [Originality](#) subsection.

6. Assessment Criteria

Your submission is assessed against the following criteria:

- Demonstrated ability to create basic SVG elements using D3 [0.5%]
- Demonstrated ability to link data to visual properties [1%]
- Demonstrated ability to create an interactive visualisation in D3 [1.5%]
- Demonstrated ability to choose appropriate visual variables to encode data [1.5%]
- Demonstrated ability to critically reflect on using different tools for creating visualisations [0.5%]

7. How to Submit

You must submit **two separate files** to Moodle:

1. A ZIP file containing only your modified `template.html`. Name the ZIP file `[STUDENT ID]_[FIRST NAME]_[LAST NAME]_PE3.zip`. Do not include any other files in this ZIP.
2. A PDF (**not** inside the ZIP — upload it as a separate file) containing:
 - Your reflection (see Section 5).
 - An originality declaration covering the sources and tools you used, in line with the [Originality](#) subsection. If you used a Generative AI tool, name it, describe what you used it for, and include a shared chat link or the list of prompts. **If you did not use a Generative AI tool, state that explicitly.**

Before submitting, check that your visualisation runs when you open `template.html` in Chrome, Firefox, or Edge. **Make sure you are submitting your completed file with your code filled in, not the original empty template.**

8. Late Penalty

As per Monash policy, all late submissions receive a deduction of 5% per day, including weekends. Work submitted more than seven days after the due date will not be marked.